Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Fran Babić, Toni Brajko, Ivan Janjić, Martina Licul, and Dorijan Lendvaj.

Implementation examples are given in attached source code files.

Task Bingo

Prepared by: Martina Licul
Necessary skills: for, matrices

Let's first solve the first subtask for $n = 1$, i. e. there is only one player. For each drawn number, we will check if it is on the player's board. If it is on the board, we will replace it with 0, since 0 cannot be drawn due to the constraints of the task. After marking all drawn numbers, we will check if the player has a *Bingo!*. We just need to check if all numbers in the row, column or diagonal are 0.

To solve the whole task, we can easily extend the solution for the first subtask to multiple players.

Task Knjige

Prepared by: Vito Anić
Necessary skills: prefix sums

It will always be optimal for all books that Marko reads (the whole book) to be adjacent. Why? Let us presume they are not adjacent. Then there would exist a place where Marko read book i and have read the covers of book $i+1$. He will definitely not reduce his inspiration if he read book $i+1$ instead of book i.

From that, we can conclude that the solution will always have first a few books that Marko has read only covers of, and then a few that he has read whole. If he skipped first x books, he can maximally read next $y = \lfloor \frac{n-b \cdot x}{a} \rfloor$ books. To determine his inspiration we can use prefix sums. We can do this for every possible x. Total time complexity: $O(n)$.

Task Lepeze

Prepared by: Ivan Janjić
Necessary skills: ad hoc, combinatorics, modular arithmetic, segment tree

Firstly, a few remarks: n is the number of vertices in a polygon. $x <-> y$ represents a diagonal with its endpoints in vertices x and y. To avoid some edge cases, sides of the polygon are regarded as diagonals.

Basic observation is that removing a diagonal uniquely determines which diagonal must be added.

Let's focus on a particular, arbitrary triangulation that, without loss of generality, we are trying to transform into a fan triangulation at vertex 1.

Lemma 1. If d diagonals have an endpoint in vertex 1, number of operations needed is $n-3-d$.

$n-3-d$ is obviously a lower bound. If $n = d-3$, we are done. Otherwise, there exists a vertex x such that diagonal $1 <-> x$ is not present. Let lo be the largest integer less than x such that there exists a diagonal $1 <-> lo$. Observe that such an integer exists as diagonal $1 <-> 2$ exists. Similarly, let hi be the smallest integer larger than x such that diagonal $1 <-> hi$ exists. Observe that such an integer exists as diagonal $1 <-> n$ exists. Claim is that diagonal $lo <-> hi$ exists. Namely, it is known that a every triangulation of every polygon has at least two "ears", where an "ear" is a vertex if it is an endpoint of no diagonal. Now it is clear, by the way we defined numbers lo and hi, that vertex 1 is an "ear" in a polygon with vertices $1, lo, lo+1, \ldots, hi-1, hi$. If a vertex is an "ear", there exists a diagonal between its neighbors, namely between lo and hi. Let's look at diagonals from vertex lo. Let mid be the largest integer less than hi such that diagonal $lo <-> mid$ exists. Such integer exists as $lo < x < hi$ and

diagonal $lo <-> lo+1$ exists. Analogously, there exists a diagonal $mid <-> hi$. Now it is clear that by removing diagonal $lo <-> hi$ we must add diagonal $1 <-> mid$, that is it is possible to increase number of diagonals with one of its endpoints in vertex 1 and we are done.

Let lo, hi and mid be integers from previous paragraph. By "fixing" diagonal $1 <-> mid$, (at most) two new intervals were created, namely $[lo, mid]$ and $[mid, hi]$ that have the same characteristics of an interval $[lo, hi]$, such as having a unique diagonal that is possible to "fix". Observations so far motivate the definition of the following graph: vertices are diagonals thast need to be "fixed" together with a dummy vertex. At the start, some diagonals are immediately "fixable" so we add a directed edge from dummy vertex to these diagonals. When we "fix" one diagonal, we add a directed edge to new diagonals that became "fixable". We created a directed acyclic graph of dependencies between diagonals. If there is a path from vertex a to vertex b, it means that diagonal a must be "fixed" before diagonal b. Also, the graph is a rooted tree with root being the dummy vertex and we are looking for the number of topological orders of this tree. Next lemma proves useful with calculating this number:

Lemma 2. Given a rooted tree with n vertices where i-th vertex has its subtree size sz_i, number of topological order is $\dfrac{n!}{\prod_{i=1}^{n} sz_i}$.

This is a well-known fact, and as such, left as an exercise for the reader. For further information refer to the following Codeforces blog: link.

To efficiently calculate the product in the denominator, lets take a step back. We associate a diagonal that must be removed with the diagonal that it "fixes". Using previous notation, we associate the diagonal $1 <-> mid$ with the diagonal $lo <-> hi$. But, by the way we chose vertices lo and hi, every vertex between them does not have a diagonal with its endpoint in vertex 1 and every such vertex corresponds to a diagonal in the subtree of the diagonal $1 <-> mid$. Observing one diagonal $x <-> y$ $(x < y)$, we see that it contributes to vertices $x+1, x+2, \ldots, y-2, y-1$ with a factor of $\frac{1}{n-(y-x)-1}$, and to all other vertices (other that x and y) with a factor of $\frac{1}{y-x-1}$. If we use a segment tree that keeps a product of numbers in an interval, the operation of removing one and adding another diagonal is a few updates in a segment tree with complexity $O(\log n)$ if we precompute inverses. We calculate the number of diagonals from each vertex explicitly and precompute needed factorials. Total time complexity is $O(n + q \log n)$ or $O((n+q)\log n)$ depending on complexity of the precomputation of inverses.

Task Putovanje

Prepared by: Toni Brajko
Necessary skills: breadth-first search (bfs), bitset

To check if a hotel can be located in a particular node (city), it is sufficient to calculate the minimum distances from that node to all others using the *breadth-first search* algorithm and then compare the obtained distances with the given distances from the task. If they match (except where $d_i = -1$), the hotel can be located in that node. The complexity of this check is $\mathcal{O}(n+m)$.

In the first and third subtasks, we check for each node in the same way whether a hotel can be located in it. The overall complexity is $\mathcal{O}(n \cdot (n+m))$.

In the second subtask, we are looking for nodes that are at a distance of d_1 from city 1. A special case is when all $d_i = -1$, in which case the hotel can be in any node. We only run *breadth-first search* for one city, so the complexity is $\mathcal{O}(n+m)$.

In the final subtask, we run *multisource BFS*. Instead of calculating distances for each node separately, we compute them simultaneously for all nodes. The idea is to calculate, for each node i, h_i - the distance of that node from the hotel, and $S_i = \{x \mid h_i + d(i,x) = d_x \neq -1\}$ where $d(x,i)$ is the distance between nodes x and i. In simple terms, S_i is the set of all nodes x from which we could obtain the given h_i (those

where distances h_i and $h_x = d_x$ are not contradictory). The hotel can only be in nodes for which $h_i = 0$ and $S_i = \{x \mid d_x \neq -1\}$.

Starting from the node with the largest distance d_i, we perform a breadth-first search, simultaneously adding nodes whose distance from the hotel equals the distance of the node in the current iteration of the algorithm. If, from node i, we observe an edge to a node j that we have already visited, and $h_j + 1 = h_i$, we add all nodes from set S_i to set S_j.

Set S_i can have a size of n, so the complexity is not significantly improved. However, notice that *bitset* supports all necessary operations! The overall complexity then becomes $\mathcal{O}(m \cdot \frac{n}{64})$, and it is sufficient to solve the entire task.

You can find more about this structure at the link. For implementation details, refer to the source code in the attached files.

Task Roboti

Prepared by: Dorijan Lendvaj
Necessary skills: depth-first search (dfs)

For $k = 0$, just answer 0 if $x_1 = x_2$ or $y_1 = y_2$ and -1 otherwise.

For $n, m \leq 300, q \leq 10$, for each query we can simulate a path in each direction from (x_1, y_1) and see if it reaches (x_2, y_2) and count turns along the way. It might end up in an infinite loop; we can force it to stop after 10^6 moves, because it's impossible for the distance to (x_2, y_2) to be that high.

For $n, m \leq 300$, we can imagine a graph where each pair of (x, y) and a direction is considered a vertex, and there is an edge from one vertex to another if, after going from (x, y) in the direction given by the initial vertex, our next move will be going from (x, y) of the next vertex in the direction given by the next vertex. Notice that each vertex has outdegree 1 and outdegree 1, so we have a graph which is a union of cycles. Now we can easily see if 2 vertices are reachable from one another by checking if they're in the same cycle, and also find the number of turns between them. We can do that for all 4 vertices that go out of (x_1, y_1) and all 4 vertices that go into (x_2, y_2) and see which pair gives the smallest number of turns.

For the full solution, notice that, outside of the case where $x_1 = x_2$ and row x_1 doesn't have anything in it or $y_1 = y_2$ and column y_1 doesn't have anything in it, the only (x, y) that you need to make vertices for are the k given (x, y). With that, finding the minimum number of turns becomes quite easy, as every vertex in the cycle represents one turn.